

SISTEMA DE GAMEPLAY, FÍSICAS Y CONTROLADOR DE IA TÁCTICA

Documentación Técnica de Desarrollo e Integración en Unreal Engine 5

Desarrollador: Alejandro Larco

Perfil del Proyecto: Demo técnica de mecánicas de combate e Inteligencia Artificial

Motor y Entorno: Unreal Engine 5 (Build Graybox)

Hardware de Pruebas: Intel Core i5-13420H | 16 GB RAM | NVIDIA RTX 4050 (6 GB VRAM)

Tecnologías Clave: C++ (Unreal Engine Framework), Blueprint Visual Scripting, Chaos Physics, Niagara VFX, Behavior Tree / Blackboard, AI Perception, NavMesh

Estado del Documento: Consolidado — Cierre de Módulo A (Fase 3)

Fecha: Septiembre 2026

ÍNDICE GENERAL

1. INTRODUCCIÓN Y ALCANCE TÉCNICO

1.1. Propósito de la demo y enfoque de desarrollo incremental

1.2. Restricciones de hardware y entorno de pruebas (Graybox)

1.3. Matriz de fases y criterios de aceptación completados

2. ARQUITECTURA DE GAMEPLAY BASE Y MOVILIDAD (FASE 1)

2.1. Sistema de balística por trazado de rayos y cadencia asíncrona

2.2. Locomoción táctica y sistema modular de parkour (Vaulting / Mantle)

2.3. Sistema de cobertura procedural dinámica ("Pop-up from Cover")

2.4. Desacoplamiento de daño localizado por hueso (BPC_Salud)

3. FEEDBACK SENSORIAL, FÍSICAS CHAOS Y RENDERIZADO (FASE 2)

3.1. Gunplay avanzado: Balanceo trigonométrico (Aim Sway) y retroceso aditivo

3.2. Sistema de desmembramiento procedural y geometría de muñones (Chaos)

3.3. Transferencia cinética, mezcla de animaciones (Hit React) y ciclo de muerte

3.4. Renderizado de impacto: Decals DBuffer y sistemas de partículas Niagara

3.5. Optimización de colisiones y suspensión de cuerpos rígidos (Rigid Body Sleep)

4. MOTOR DE INTELIGENCIA ARTIFICIAL TÁCTICA (FASE 3 — MÓDULO A)

4.1. Arquitectura híbrida: Controlador C++ (AAIController) y lógica en Blueprints

4.2. Infraestructura de percepción sensorial (UAIPerception: Vista, Oído y Daño)

4.3. Estructura de memoria cognitiva en tiempo de ejecución (Blackboard)

4.4. Árbol de comportamiento (Behavior Tree) y flujo de prioridades

4.5. Navegación dinámica en el nivel y enlaces verticales (NavMesh y NavLinkProxy)

4.6. Rastreo continuo (BTS_FijarBlanco) y control de cadencia en IA

4.7. Vigilancia procedural: Barrido visual y control de rotación por punto focal

5. REGISTRO DE PROBLEMAS RESUELTOS Y OPTIMIZACIÓN (DEBUGGING)

5.1. Control de recursos: Corrección de fuga de memoria en Event Tick (Spring Arm)

5.2. Colisiones y jerarquía: Corrección de Capsule Riding e interferencias en Ragdoll

5.3. Renderizado y proyección: Supresión de Z-Fighting y confinamiento de Decals

5.4. Concurrencia y sincronización en IA: Prevención de llamadas Accessed None

6. ESTADO ACTUAL Y PRÓXIMOS PASOS

6.1. Hitos completados del Módulo A

6.2. Roadmap Fase B: Environment Query System (EQS), coberturas y flanqueo táctico

1. INTRODUCCIÓN Y ALCANCE TÉCNICO

1.1. Propósito de la demo y enfoque de desarrollo incremental

El presente proyecto consiste en el desarrollo de una demo técnica en Unreal Engine 5 orientada a la implementación, desacoplamiento y validación de tres pilares de jugabilidad en títulos de acción en tercera persona: mecánicas de control y balística matemática, simulación de físicas y desmembramiento mediante el motor Chaos, y un controlador de Inteligencia Artificial (IA) táctica para entidades no jugables (NPC).

El desarrollo se planificó bajo un enfoque estrictamente incremental y modular, aplicando las siguientes pautas de trabajo:

- **Aislamiento funcional antes de la representación visual:** La lógica de cálculo (trayectorias de disparo por trazado de rayos, estados de cobertura y transiciones de movimiento) se construyó y validó matemáticamente de forma previa a la integración de animaciones, efectos visuales o mallas definitivas.
- **Separación de responsabilidades:** La arquitectura desacopla el cerebro lógico (controladores de IA y componentes de salud) de la representación física y visual de las entidades, facilitando la extensión de nuevas armas o comportamientos sin modificar el código base.
- **Cierre de fases por criterios de aceptación:** Ningún subsistema avanzó a la siguiente etapa sin superar pruebas de estrés funcionales (ausencia de fugas de memoria, control de colisiones erráticas y ejecución de secuencias asíncronas sin bloqueos).

1.2. Restricciones de hardware y entorno de pruebas (Graybox)

Para garantizar que las mecánicas sean estables y escalables, el proyecto se probó y calibró bajo una configuración de hardware estándar de gama media:

- **Procesador (CPU):** Intel Core i5-13420H.
- **Memoria RAM:** 16 GB DDR5.
- **Tarjeta Gráfica (GPU):** NVIDIA GeForce RTX 4050 Laptop GPU (6 GB VRAM).
- **Almacenamiento:** Unidad de estado sólido (SSD).

Uso del entorno Graybox

Las pruebas se ejecutaron deliberadamente sobre un nivel prototipo (*Graybox*). Prescindir de iluminación horneada pesada, geometrías complejas o texturas de alta resolución permitió aislar el consumo de cómputo del procesador y la tarjeta gráfica. De este modo, cualquier caída de fotogramas, desincronización en la cola de tareas o conflicto de memoria se atribuyó y corrigió directamente sobre el código de la lógica, el trazado de rayos (*line traces*) o la simulación de cuerpos rígidos, sin factores externos de renderizado que enmascaren errores de diseño.

1.3. Matriz de fases y criterios de aceptación

El proyecto consolidó las primeras tres fases de desarrollo técnico, cerrando formalmente el Módulo A de la Inteligencia Artificial:

Fase	Denominación	Estado	Validación Técnica Principal
Fase 1	Gameplay Base y Matemáticas	Completada	Balística por <i>Line Trace</i> , cadencia gobernada por temporizadores, cobertura procedural por raycasting y daño localizado por hueso mediante diccionarios (<i>Map</i>).
Fase 2	Físicas Chaos y Feedback Sensorial	Completada	Retroceso procedimental aditivo, desmembramiento con mallas polimórficas, decals proyectados en DBuffer y optimización de cuerpos caídos mediante <i>Rigid Body Sleep</i> .
Fase 3 (Módulo A)	Controlador de IA y Ciclo de Combate	Completada	Arquitectura híbrida C++/ <i>Behavior Tree</i> , doble percepción (visión/oído/daño), tracking continuo del objetivo (<i>Set Focus</i>) y barrido visual procedural.

Fase	Denominación	Estado	Validación Técnica Principal
Fase 3 (Módulo B)	Sistemas de Consulta Ambiental (EQS)	Próximo	Integración de <i>Environment Query System</i> para búsqueda dinámica de coberturas, rutas de flanqueo inteligente y afiliación por bandos (<i>Team IDs</i>).

2. ARQUITECTURA DE GAMEPLAY BASE Y MOVILIDAD (FASE 1)

El objetivo de la Fase 1 consistió en estructurar las bases funcionales y matemáticas del personaje jugable y de los objetivos en el mundo, priorizando la precisión numérica y el desacoplamiento lógico antes de integrar efectos visuales complejos o animaciones definitivas.

2.1. Sistema de balística por trazado de rayos y cadencia asíncrona

El sistema de armamento se diseñó bajo una jerarquía orientada a objetos (POO) para facilitar su escalabilidad y mantenimiento:

- **Jerarquía de clases:** Se estableció una clase maestra (BP_ArmaBase) que centraliza el grafo de eventos y las variables universales (daño base, cadencia, capacidad de cargador y lógica de disparo). Las armas específicas (por ejemplo, la pistola de gran calibre) se crearon como clases hijas derivadas, modificando únicamente sus valores por defecto en el panel de configuración sin alterar el código base.
- **Instanciación dinámica:** El personaje actúa como un controlador agnóstico respecto al arma. Durante el evento de inicialización (BeginPlay), el arma se instancia en el mundo mediante `SpawnActorFromClass` y se acopla dinámicamente al socket correspondiente de la malla esquelética (`AttachActorToComponent`).
- **Balística por trazado de rayos (*Hitscan*):** Se descartó el uso de proyectiles físicos basados en el *Tick* del motor para optimizar el consumo de procesamiento y asegurar registro inmediato de impactos a distancias cortas y medias. El cálculo se realiza en dos etapas:
 1. Se proyecta un rayo desde el centro del *Viewport* de la cámara para identificar el punto exacto hacia donde mira el jugador.

2. Se traza un segundo vector desde la boca del cañón hacia las coordenadas del impacto del primer rayo, extendiéndose con una longitud escalar de 10.000 unidades. Esto asegura que la trayectoria atraviese la cápsula de colisión general e interactúe directamente con los huesos de la malla física.
- **Control de cadencia asíncrono:** Para evitar la acumulación de instrucciones o retrasos rígidos por fotograma, se eliminaron los nodos de pausa (Delay). La cadencia de disparo se gestiona mediante un temporizador asíncrono (Set Timer by Event) vinculado a una compuerta booleana (Puede_Disparar). Las pulsaciones repetitivas del gatillo mientras el arma está en enfriamiento se descartan de forma silenciosa sin encolar eventos.
 - **Sincronización de recarga:** El flujo de munición queda bloqueado por la variable de estado EstaRecargando?. La actualización matemática del cargador se vincula al pin de finalización (On Completed) del montaje de animación (Play Montage), evitando desincronizaciones si la animación se interrumpe.
 - **Dispersión angular (*Aim Sway*):** El balanceo pasivo del arma se calculó mediante funciones trigonométricas continuas (seno y coseno) dependientes del tiempo de juego. Al aplicarse sobre el espacio de coordenadas locales de la vista, se evita alterar las rotaciones de cámara del jugador, preservando la alineación geométrica del retículo.

2.2. Locomoción táctica y sistema modular de parkour (Vaulting / Mantle)

La máquina de estados de movimiento desacopla el cálculo espacial de los obstáculos de las animaciones visuales, manteniendo la estabilidad de la cápsula física en todo momento:

- **Manipulación de la cápsula y postura:** Se integraron las funciones nativas Crouch y Uncrouch del componente de movimiento, las cuales ajustan dinámicamente el parámetro Crouched Half Height para permitir el paso por aberturas reducidas. Para permitir apuntar verticalmente en cuclillas sin deformar la animación de la cadera, se aplica una rotación procedural sobre la espina dorsal (Transform Modify Bone) gobernada por el ángulo vertical de la vista (Aim Pitch).
- **Detección de geometría y enrutamiento modular:** Mediante un trazado de rayos por canal (Line Trace For Objects) configurado exclusivamente para el canal personalizado ObstaculoParkour, el sistema detecta la altura del muro ignorando otras entidades como NPCs. La coordenada vertical **Z** del punto de impacto se clasifica por rangos numéricos:
 - **Altura menor a 90 cm:** Se ejecuta un salto ágil de apoyo (*Mantle*).

- **Altura entre 90.1 cm y 125 cm:** Se dispara una acción de trepa completa con mayor tiempo de suspensión.
- **Desplazamiento por curvas interpoladas (*Timeline*):** En lugar de forzar la posición de la cápsula con saltos instantáneos, el desplazamiento se controla mediante una curva *Timeline* asíncrona. Esta utiliza una función de campana para el eje vertical **Z** y una curva sigmoide (S-Curve) para el plano horizontal **XY**. Se introdujo un retardo operativo de 0.33 segundos para sincronizar el despegue físico con el contacto de las manos en la animación.
- **Transición de modos de movimiento:** Durante el salto, el modo de desplazamiento cambia temporalmente de Walking a Flying, desactivando la comprobación continua de colisión lateral (Sweep). Para evitar que la cápsula quede atascada dentro de la geometría al reactivar las físicas en el punto de llegada, se añade un desfase de seguridad de +15.0 unidades en el eje vertical.
- **Estructura del Blueprint de Animación (*AnimBP*):** Para reducir la carga de CPU, la pose de locomoción base se evalúa una sola vez por fotograma y se almacena en memoria caché (*Cached Poses*). Mediante el nodo Layered blend per bone (seccionando desde spine_01 o clavicle_l), se independizan las acciones de disparo o recarga del tren superior respecto a la locomoción inferior. Asimismo, un trazado frontal continuo desactiva el apuntado frente a muros cercanos, forzando la transición a postura de guardia baja (*Low Ready*).

2.3. Sistema de cobertura procedural dinámica ("Pop-up from Cover")

Se diseñó una mecánica para permitir al jugador asomarse por encima de coberturas bajas al apuntar y regresar a posición protegida al soltar el gatillo, sin requerir pulsaciones dobles de teclado:

- **Detección espacial frontal:** Al iniciar la acción de apuntado, un nodo Line Trace By Channel proyecta un vector de 100 unidades hacia adelante utilizando el vector frontal del personaje (Forward Vector) en el canal Visibility. Esto confirma si el personaje se encuentra físicamente adyacente a una estructura de cobertura baja antes de alterar la postura.
- **Gestión de memoria de estado:** Se implementó una variable booleana interna (EstabaEnCobertura). Si el sistema confirma que el personaje está agachado y con un obstáculo al frente, ejecuta Uncrouch y establece la variable en verdadero. Al liberar el gatillo de apuntado, la lectura de esta bandera dispara automáticamente la función Crouch y restablece la variable a falso, asegurando el retorno a cobertura.
- **Interpolación suave de cámara:** Para mitigar vibraciones o cortes visuales bruscos provocados por la compresión y descompresión física de la cápsula, la cámara se desacopló del eje de la raíz. La altura de la cámara interpola de

manera continua entre -35.0 (agachado) y +3.52 unidades (de pie) mediante la función matemática `FInterp To` en el ciclo principal (Tick).

- **Criterio de integración nativa:** Al utilizar las funciones del motor (`Crouch/Uncrouch`) en lugar de deformaciones de escala personalizadas, se preserva la compatibilidad nativa con las máquinas de estados del *AnimBP*, el cual simplemente lee el estado booleano de locomoción para mezclar los espacios de interpolación (*Blendspaces*).

2.4. Desacoplamiento de daño localizado por hueso (BPC_Salud)

Para los objetivos y futuros personajes no jugables, se estructuró un sistema modular de recepción de daño basado en componentes:

- **Componente independiente (BPC_Salud):** Diseñado bajo el principio de responsabilidad única como un componente de actor (`ActorComponent`), este módulo gestiona la integridad física de la entidad de manera autónoma, aislando el cálculo numérico de la malla esquelética y de los árboles de comportamiento.
- **Estructura de datos por extremidad:** La salud no se limita a un único valor escalar flotante. El componente utiliza un diccionario (Map con clave `FName` y valor `Float`) que almacena la resistencia individual de cada zona corporal. Al registrarse un impacto, el sistema extrae el identificador del hueso desde el resultado del trazado de rayos (`Hit Bone Name`) y descuenta el daño correspondiente a dicha extremidad de forma independiente a la salud general.
- **Multiplicadores críticos (Headshots):** Cuando el trazado balístico reporta impacto en la zona de la cabeza, un nodo de selección condicional (`Select Float`) aplica un multiplicador de daño crítico sobre el daño base del arma atacante, permitiendo la neutralización inmediata del objetivo sin requerir flujos de código separados.

3. FEEDBACK SENSORIAL, FÍSICAS CHAOS Y RENDERIZADO (FASE 2)

La Fase 2 se enfocó en dotar de peso físico y respuesta sensorial a las mecánicas matemáticas construidas en la Fase 1. El objetivo fue implementar efectos visuales, dinámicas corporales ante impactos de alta energía y desmembramientos en tiempo real, priorizando la optimización del rendimiento en CPU y GPU.

3.1. Gunplay avanzado: Balanceo procedimental (Aim Sway) y retroceso aditivo

El comportamiento del arma y la cámara se reconstruyó para transmitir inercia física sin comprometer la puntería geométrica:

- **Balanceo táctico desacoplado (*Aim Sway*):** Se descartaron los sacudones de cámara tradicionales (*Camera Shakes*) para evitar distorsiones visuales en el campo de visión. En su lugar, se implementó un balanceo dinámico basado en funciones trigonométricas (seno y coseno) impulsadas por el tiempo de juego (Get Game Time in Seconds).
- **Control de consumo de recursos:** Para evitar cálculos continuos innecesarios, la lógica matemática se condicionó mediante una compuerta booleana (*EstaApuntando?*) en el ciclo principal (Tick), limitando la ejecución exclusivamente a la fase de puntería activa.
- **Sincronización cinemática con el modelo:** Mediante variables puente (*Sway_Pitch* y *Sway_Yaw*), las rotaciones matemáticas se inyectan en el grafo de animación (*AnimGraph*). Mediante el nodo Transform Modify Bone, se aplican rotaciones locales aditivas sobre el hueso *spine_03*, logrando que el torso del personaje y el arma sigan el movimiento del retículo de forma orgánica.
- **Retroceso híbrido procedimental:** La respuesta al disparo se estructuró en dos capas simultáneas para no interrumpir el desplazamiento:
 - **Capa de animación:** Se aisló una secuencia de disparo asignada a un montaje en el slot *UpperBody*, superponiendo el culatazo en el tren superior sin congelar la locomoción de las piernas.
 - **Capa física en cámara:** Se inyectó fuerza rotacional mediante Add Controller Pitch/Yaw Input, distribuyendo la aceleración a través de una curva matemática dentro de un nodo Timeline de 0.1 segundos para amortiguar el retroceso visual.
- **Efectos de boca de cañón (*Muzzle Flash*):** Se optimizó la emisión visual anclando un emisor de partículas Cascade directamente al socket de la boca del arma (*Muzzle*), aplicando un factor de escala reducido (0.05) para ofrecer retroalimentación visual clara sin obstaculizar la visión del jugador.

3.2. Sistema de desmembramiento por datos y geometría modular (*Chaos Physics*)

Para evitar el uso de plugins externos de corte procedimental que comprometen el rendimiento del procesador, se diseñó una solución nativa basada en estructuras de datos y reemplazo de mallas estáticas:

- **Actor polimórfico de extremidades (*BP_ExtremidadFisica*):** Se implementó una clase base para gestionar cualquier fragmento mutilado. Al instanciarse (*SpawnActor*), el actor recibe el identificador del hueso seccionado (*HuesoAmputado*) y, mediante un árbol condicional Switch on Name, asigna la malla física y las propiedades colisionables adecuadas en el primer fotograma.

- **Optimización de mallas y texturas orgánicas:** Las secciones anatómicas compuestas se unificaron utilizando las herramientas de modelado de Unreal Engine 5 (*Merge XForm*). Esto generó mallas estáticas optimizadas (como SM_BrazoMutilado) que reducen las llamadas de dibujo (*draw calls*). En el material, se ajustó el tinte de albedo hacia tonos carmesí y se limitó la rugosidad máxima (*Roughness Max*) a 0.3 para reflejar especularidad húmeda simulando tejido orgánico.
- **Diseño guiado por datos (*Data-Driven*):** Se creó la estructura ST_DatosMuñon, que almacena por cada sección anatómica el hueso de anclaje, la referencia a la malla estática de cierre y las transformaciones espaciales locales. En el Blueprint del enemigo, un diccionario (Map llamado HuesosKey) asocia cada hueso destructible con su estructura correspondiente, manteniendo la lógica visual independiente del componente numérico BPC_Salud.
- **Ensamblaje dinámico del muñon:** Tras registrarse una amputación, el sistema consulta el diccionario, instancia la geometría requerida con Add Static Mesh Component y la acopla al esqueleto con Attach Component To Component en modo *Snap to Target*, asegurando que el muñon conserve la orientación y las físicas simuladas del motor Chaos durante el estado inerte (*Ragdoll*).
- **Fragmentación por impactos críticos:** Para disparos de alto calibre o impactos letales directos, un bucle de ejecución instancia de tres a cuatro fragmentos adicionales dispersados con el nodo Random Unit Vector in Cone in Degrees (cono de dispersión de 45°). Estos elementos se enrutan bajo la categoría Gib para restringir su volumen y evitar anomalías visuales.

3.3. Cinemática de impacto, animaciones aditivas (*Hit React*) y flujo de muerte

La respuesta física de los impactos se desacopló de las secuencias rígidas de animación para lograr reacciones corporales más predictivas:

- **Reacción aditiva a impactos (*Hit React*):** Mediante el nodo Layered blend per bone aplicado a la raíz del torso (spine_01) y filtrado por un selector Switch on Name, se reproducen animaciones de dolor localizadas según la zona impactada sin interrumpir el ciclo de locomoción inferior del NPC.
- **Transferencia de momento lineal:** Se calculó el impulso cinético multiplicando el vector unitario de impacto (Get Unit Direction) por un factor escalar de masa (9.000 unidades). Esta magnitud se transfiere vectorialmente al punto geométrico exacto de colisión a través de Add Impulse At Location.
- **Flujo modular de transición a Ragdoll:** Para evitar duplicidad de código y ejecuciones concurrentes erróneas, se centralizó la secuencia de muerte en el evento unificado EncenderFisicasBase:

1. Cambio de perfil de colisión a Ragdoll.
 2. Bloqueo del canal Visibility para suspender el registro de nuevos disparos sobre el objetivo abatido.
 3. Activación de simulación de físicas (Set Simulate Physics) en la malla esquelética.
 4. Desactivación del componente de colisión (Capsule Component) para suprimir obstáculos fantasma en la navegación del nivel.
- **Sincronización de estado vital:** El componente BPC_Salud evalúa de forma síncrona si los puntos de vida llegan a cero. Antes de notificar al actor principal, invalida cualquier temporizador de daño periódico o sangrado mediante Clear and Invalidate Timer by Handle, asegurando un cierre de ciclo libre de fugas lógicas.

3.4. Renderizado volumétrico y fluidos (Decals DBuffer y sistemas Niagara)

El sistema de proyección de fluidos y partículas en el entorno se diseñó para ofrecer retroalimentación de impacto sin comprometer el renderizado estático:

- **Alineación ortogonal de manchas (Decals):** A partir del vector normal de superficie (Impact Normal) obtenido en el impacto del trazado de rayos, se calcula la rotación del decal proyectado, garantizando que el material quede perfectamente alineado con la orientación de la superficie.
- **Corrección de iluminación indirecta (DBuffer):** Para evitar que las manchas se rendericen oscuras en áreas bajo sombra, se activó la compatibilidad con *DBuffer Decals* en las configuraciones del proyecto, asignando el modo de mezcla a DBuffer Translucent Color, Normal, Roughness. Asimismo, se igualaron paramétricamente las dimensiones laterales de proyección para prevenir deformaciones visuales (*stretching*).
- **Sistemas de partículas Niagara:** Utilizando el mismo vector normal del impacto, se orienta la dirección de emisión del emisor de partículas de sangre, logrando que el fluido brote en correspondencia física con el ángulo de penetración balística.

3.5. Optimización de colisiones y suspensión de cuerpos rígidos (Rigid Body Sleep)

Para garantizar una tasa de fotogramas estable en equipos de especificaciones medias, se implementaron salvaguardas arquitectónicas sobre la simulación física:

- **Aislamiento de colisiones en fragmentos cárnicos:** Para neutralizar anomalías físicas donde los fragmentos desprendidos interferían con la cápsula

del personaje haciéndolo flotar o tropezar (*Capsule Riding*), se configuró un perfil de colisión personalizado. La malla de los fragmentos ignora (Ignore) los canales de entidades móviles (Pawn, Camera) y mantiene bloqueo estricto (Block) únicamente frente al entorno estático y dinámico (WorldStatic, WorldDynamic).

- **Suspensión de cálculo de CPU (*Rigid Body Sleep*):** Se implementó una compuerta booleana de estado (YaEstaMuerto?) para neutralizar ciclos repetitivos. Tras el colapso del cuerpo en *Ragdoll*, un temporizador asíncrono de 4.0 segundos ejecuta la función nativa Put Rigid Body to Sleep (con el parámetro Bone Name en None). Esto instruye al motor Chaos a suspender por completo las operaciones de gravedad, fricción e inercia sobre el esqueleto en reposo, reduciendo el consumo de cómputo del cadáver a cero sin retirarlo visualmente de la escena.

4. MOTOR DE INTELIGENCIA ARTIFICIAL TÁCTICA (FASE 3 — MÓDULO A)

El desarrollo de la Fase 3 se centró en la creación de un sistema de Inteligencia Artificial (IA) para entidades no jugables (NPC), culminando en el Módulo A con la integración completa del ciclo de percepción, memoria a corto plazo, toma de decisiones jerárquica y combate táctico en un entorno de pruebas *Graybox*.

4.1. Arquitectura híbrida: Controlador C++ (AAIController) y lógica en Blueprints

Para equilibrar el rendimiento computacional con la flexibilidad de diseño, se adoptó una arquitectura híbrida dividida entre código C++ y secuencias visuales de Blueprint:

- **Controlador lógico nativo (ControladorEnemigoBase):** Se implementó una clase en C++ heredando de la clase base AAIController del Gameplay Framework de Unreal Engine. Esta separación desacopla por completo la toma de decisiones y el procesamiento de sentidos de la malla esquelética (BP_Enemigo_Definitivo), permitiendo reutilizar el mismo cerebro lógico en diferentes variantes de personajes o modelos 3D sin duplicar código.
- **Configuración del entorno de desarrollo y dependencias:** El entorno se configuró en Visual Studio 2022 con los módulos requeridos para el compilador MSVC y el soporte de Unreal Engine. Para habilitar el acceso a los subsistemas de IA avanzados en C++, se modificó el archivo de compilación del proyecto (MyProject1.Build.cs), incorporando explícitamente las dependencias de los módulos AIModule y GameplayTasks.
- **Secuencia de inicialización en tiempo de ejecución:** Para mitigar condiciones de carrera (*Race Conditions*) que generaban errores de acceso nulo (Accessed None), se trasladó la ejecución del árbol de comportamiento (Run Behavior Tree) fuera del evento BeginPlay del personaje. La inicialización se delegó al

evento nativo OnPossess dentro de la clase derivada BP_CerebroEnemigo, garantizando que el árbol solo se active una vez que el controlador tenga posesión válida y confirmada del peón en el mundo.

4.2. Infraestructura de percepción sensorial (UAIPerception: Vista, Oído y Daño)

El procesamiento de estímulos del entorno se programó a nivel nativo en C++ utilizando el componente UAIPerceptionComponent acoplado al controlador:

- **Configuración visual (UAISenseConfig_Sight):** Se definieron radios de detección comprendidos entre 1.500 y 2.000 unidades espaciales. El sistema evalúa el cono de visión y calcula el producto punto (*Dot Product*) respecto al vector frontal, diferenciando matemáticamente la visión periférica de la visión central para aplicar tiempos de respuesta más rápidos cuando el objetivo se expone de frente.
- **Configuración auditiva (UAISenseConfig_Hearing):** Se configuró un radio sensorial de 2.500 unidades. El sistema escucha eventos de sonido emitidos mediante Report Noise Event (acoplados al disparo del arma del jugador), permitiendo al NPC orientarse e investigar alertas acústicas fuera de su campo de visión directo.
- **Percepción de daño (UAISense_Damage):** Se programó el sentido de daño dentro de la clase nativa para responder a ataques balísticos de larga distancia donde el agresor no es visible, asegurando que el NPC salga del estado de calma e inicie de inmediato maniobras de cobertura o búsqueda.
- **Filtrado de estímulos y aislamiento de fuego amigo:** Dentro de la función de respuesta AIDetectarEstimulo, se implementó una validación mediante ActorHasTag("Jugador"). Esto aísla el procesamiento sensorial de modo que la entidad ignore los sonidos o la presencia de otros NPCs aliados, evitando estados de alerta cruzados o fuego amigo involuntario.

4.3. Estructura de memoria cognitiva en tiempo de ejecución (Blackboard)

La memoria de trabajo de cada entidad se estructuró a través del activo de datos BB_MemoriaEnemigo, el cual desacopla la información del estado del mundo de la lógica de evaluación:

- **Aislamiento por instancia (*Instanced Blackboard*):** Para evitar que múltiples NPCs compartieran erróneamente un mismo contexto de memoria, se desactivó la sincronización global del Blackboard, garantizando variables independientes para cada entidad en escena.
- **Diccionario de variables cognitivas:**

- JugadorVisto (*Boolean*): Estado síncrono de contacto visual con el objetivo.
- UltimaPosicionConocida (*Vector*): Coordenadas espaciales (\$XYZ\$) del último punto donde se confirmó visión activa.
- ActorJugador (*Object / Actor*): Puntero directo a la referencia del actor objetivo en memoria.
- NivelDeMoral (*Float*): Escalar dinámico que modula la transición a comportamientos defensivos.
- En_Panico (*Boolean*): Bandera de activación para rutas de huida desordenada.
- Pierna_Destruida (*Boolean*): Bandera de agonía física que restringe la velocidad y transiciones de locomoción.
- DestinoPatrullaje (*Vector*): Coordenadas asignadas para la navegación pasiva.
- **Preservación de memoria espacial:** En la función de pérdida de estímulo visual de C++, se programó la inyección directa de StimulusLocation sobre la clave UltimaPosicionConocida al momento exacto de romper la línea de visión. Esto neutraliza la amnesia espacial, forzando a la IA a investigar el punto exacto de desaparición y no la ubicación donde se produjo el primer contacto.

4.4. Árbol de comportamiento (Behavior Tree) y flujo de prioridades

La toma de decisiones reside en el árbol de comportamiento BT_CerebroEnemigo, cuya estructura jerárquica utiliza un nodo selector raíz que evalúa de izquierda a derecha seis ramas tácticas por estricto orden de urgencia:

1. **Agonía (Sequence — Prioridad 1):** Máxima urgencia. Se ejecuta cuando la bandera Pierna_Destruida es verdadera, anulando cualquier ofensiva para gestionar la inmovilidad del cuerpo.
2. **Pánico (Sequence — Prioridad 2):** Se dispara ante el colapso del NivelDeMoral, forzando la desvinculación del combate y la búsqueda de rutas de escape.
3. **Asalto (Sequence — Prioridad 3):** Ofensiva frontal cuando existe confirmación visual directa (JugadorVisto = True) y proximidad favorable.
4. **Combate Táctico (Selector — Prioridad 4):** Maniobras coordinadas de fuego sostenido, desplazamiento lateral (*strafe*) y uso de coberturas durante enfrentamientos activos.
5. **Búsqueda (Sequence — Prioridad 5):** Investigación sistemática del vector UltimaPosicionConocida tras perder la línea de visión con el objetivo.
6. **Patrullaje (Sequence — Prioridad 6):** Merodeo cíclico o patrullaje por waypoints en estado de reposo.

- **Candados condicionales (*Decorators*) y reactividad:** Las primeras cinco ramas cuentan con decoradores de Blackboard configurados con la regla Observer Aborts: Both. Esta directiva permite interrumpir secuencias activas de baja prioridad en el mismo fotograma en que una variable crítica cambia (por ejemplo, suspender de inmediato el patrullaje o la búsqueda si el jugador vuelve a entrar en el cono visual).

4.5. Navegación dinámica en el nivel y enlaces verticales (NavMesh y NavLinkProxy)

El desplazamiento autónomo en el espacio tridimensional combina volúmenes de navegación nativos con cálculos modulares:

- **Malla de navegación (*NavMesh*):** Delimitada por volúmenes NavMeshBoundsVolume para calcular trayectorias en tiempo real mediante la tarea nativa Move To.
- **Transiciones verticales (*NavLinkProxy*):** Se implementaron proxies de enlace bidireccional sobre los muros de cobertura baja, permitiendo al algoritmo de búsqueda de caminos (*Pathfinding*) calcular trayectorias de salto y trepa. Para evitar que la cápsula colisionara contra el borde de la geometría durante el salto, se fijaron los vectores extremos a las cotas transitables y se calibró la variable Max Step Height del componente de movimiento.
- **Control cinemático de velocidad (*BTT_CambiarVelocidad*):** Se desarrolló una tarea personalizada en el árbol que modula dinámicamente la velocidad máxima de desplazamiento según el estado cognitivo:
 - **200 unidades:** Exploración pasiva y patrullaje preventivo.
 - **600 unidades:** Fases de alerta confirmada, asalto y búsqueda activa.
- **Modularidad de patrullaje:** Mediante las tareas BTT_BuscarPuntoAleatorio (radio de 1.500 unidades) y BTT_ElegirPuntoPatrulla (parametrizable con variables expuestas en el editor Instance Editable), una misma clase base puede adoptar rutas fijas o deambulaciones aleatorias sin alterar la arquitectura lógica.

4.6. Ciclo de combate: Rastreo continuo (*BTS_FijarBlanco*) y control de cadencia en IA

El enfrentamiento armado requirió desacoplar la orientación de puntería de la dirección del movimiento físico:

- **Servicio de enfoque asíncrono (*BTS_FijarBlanco*):** Se descartaron tareas de rotación bloqueantes (Rotate to Face Entry) y se optó por un servicio acoplado a las ramas de combate que invoca la función nativa Set Focus del controlador.

Esto fuerza al NPC a mantener su vector frontal alineado continuamente con el jugador mientras se desplaza.

- **Desacoplamiento de locomoción (*Strafe*):** En el componente CharacterMovement del NPC se ajustaron las propiedades de rotación:
 - Orient Rotation to Movement: **Desactivado**.
 - Use Controller Desired Rotation: **Activado**.

Esta configuración otorga al controlador el mando directo sobre el eje vertical (Yaw), permitiendo que la entidad camine lateralmente o retroceda mientras mantiene el torso y el arma apuntando al objetivo.

- **Corrección de precisión en disparo:** El trazado balístico del enemigo se vinculó geométricamente al socket de salida del cañón (Muzzle Socket). Para mitigar trayectorias artificialmente perfectas (*aimbot*), se integró una dispersión angular calculada mediante el nodo Random Unit Vector in Cone in Degrees.
- **Sincronización de cadencia en el árbol:** Para evitar que la evaluación continua del nodo selector provocara disparos en cada fotograma, la tarea personalizada BTT_Disparar ejecuta una comprobación de estado sobre el arma equipada (BP_ArmaBase). Si el arma se encuentra en periodo de enfriamiento (gobernada por su temporizador asíncrono), la tarea retorna un estado de éxito (Success) sin generar trazados balísticos redundantes, obligando a la IA a respetar la cadencia mecánica nominal.

4.7. Vigilancia procedural: Barrido visual y control de rotación por punto focal

Para eliminar la rigidez estática durante las secuencias de guardia y búsqueda, se estructuró un sistema de observación activa:

- **Deprecación de rotaciones en el grafo de animación (*AnimGraph*):** Se descartó la manipulación manual de las articulaciones de la columna (spine_03) mediante el nodo Look At, debido a inconsistencias de orientación en los ejes locales de la malla esquelética que generaban deformaciones visuales y colapso de rotaciones (*Gimbal Lock*).
- **Rotación física de cápsula (Set Focal Point):** La vigilancia se trasladó al controlador lógico mediante la función Set Focal Point. Esto garantiza que la cápsula física completa y el cono de percepción sensorial giren de forma solidaria en el espacio 3D. Para evitar giros instantáneos, se ajustó el valor de Rotation Rate (Yaw) a 150.0 grados por segundo, logrando una interpolación física natural.
- **Generación aleatoria de vectores de interés:** A través de las tareas BTT_EncontrarPuntoInteres y BTT_ResetearMirada, la entidad proyecta puntos

de inspección calculados de forma relativa a su posición (Transform Location) dentro de una envolvente procedural: profundidad longitudinal de 300 a 800 unidades, amplitud lateral de -400 a 400 unidades, y elevación nula ($Z = 0.0$).

- **Estructura de bucles anidados en Behavior Tree:** Se implementó una jerarquía de doble ciclo:
 - **Bucle mayor (Macro-loop):** Gobierna el traslado físico de la entidad entre los nodos de navegación del escenario.
 - **Bucle menor (Micro-loop):** Secuencia interna gobernada por un decorador Loop que ejecuta inspecciones visuales cortas (cálculo de punto focal, pausa de observación con el nodo Wait y reseteo de orientación) antes de proseguir con el siguiente tramo de la ruta.

5. REGISTRO DE PROBLEMAS RESUELTOS Y OPTIMIZACIÓN (DEBUGGING)

Este apartado consolida los fallos críticos, advertencias de motor y anomalías de sincronización detectados a lo largo de las tres fases de desarrollo, documentando su causa raíz, la solución arquitectónica implementada y el criterio técnico adoptado.

5.1. Control de recursos y ciclo de vida

Fuga de memoria por instanciación en el ciclo principal (Event Tick)

- **Problema:** Caída progresiva y severa de la tasa de fotogramas por segundo durante la ejecución de la Fase 2.
- **Causa raíz:** En la integración inicial del balanceo de puntería (*Aim Sway*), se introdujo inadvertidamente un nodo Add Spring Arm Component dentro del flujo continuo de ejecución del Event Tick, provocando la creación de cientos de componentes de cámara redundantes por segundo en memoria.
- **Solución aplicada:** Se eliminó la instanciación dinámica en el bucle principal. El balanceo se reconfiguró interviniendo exclusivamente las variables de rotación local del Spring Arm ya existente y aplicando compensaciones aditivas sobre el hueso spine_03 en el grafo de animación (*AnimGraph*).

Sobrecarga computacional post-mortem en simulación Chaos

- **Problema:** Incremento continuo en el tiempo de procesamiento de la CPU conforme aumentaba el número de enemigos abatidos en el nivel de pruebas.
- **Causa raíz:** El motor de físicas Chaos continuaba calculando vectores de fricción, micro-colisiones y gravedad sobre los esqueletos en estado inerte (*Ragdoll*) de forma indefinida, aun cuando los cuerpos ya habían alcanzado el reposo en el suelo.

- **Solución aplicada:** Se programó un temporizador asíncrono de 4.0 segundos posterior al colapso del cuerpo. Mediante una compuerta booleana de estado (YaEstaMuerto?), se dispara la función nativa Put Rigid Body to Sleep (con parámetro Bone Name: None), suspendiendo por completo las operaciones físicas del esqueleto y reduciendo su costo de CPU a cero sin destruir la malla de la escena.

5.2. Colisiones y jerarquías esqueléticas (Chaos Physics)

Inestabilidad cinemática por fragmentos desmembrados (*Capsule Riding*)

- **Problema:** Tras seccionar extremidades, los fragmentos colisionaban de manera caótica contra la cápsula física del propio NPC o del jugador, provocando desplazamientos erráticos, elevación forzada de entidades (*Physics Jitter*) o tropiezos mecánicos.
- **Causa raíz:** Las mallas de carne y hueso generadas utilizaban el perfil de colisión por defecto PhysicsActor, colisionando con los canales Pawn y Camera.
- **Solución aplicada:** Se reconfiguró el perfil de colisión de las extremidades a Custom, forzándolas a ignorar (Ignore) los canales Pawn y Camera, y conservando bloqueo (Block) exclusivamente frente a geometría estática y dinámica del entorno (WorldStatic, WorldDynamic).

Geometría huérfana en amputaciones sucesivas (Muñones en cascada)

- **Problema:** Al desmembrar una articulación superior (por ejemplo, hombro) después de haber desmembrado una inferior (antebrazo), la malla de la tapa del muñón inferior perdía su hueso de referencia y quedaba suspendida de forma estática en el aire.
- **Causa raíz:** Ruptura de la jerarquía esquelética; el actor hijo no gestionaba la destrucción previa de los sockets descendientes en mallas compuestas.
- **Solución aplicada:** Se implementó una rutina de barrido previa a la mutilación. Mediante la etiqueta Component Tags: Munon y el nodo Bone Is Child Of, el sistema evalúa si existen componentes de muñón anclados a huesos descendientes del nuevo corte; de ser afirmativo, ejecuta Destroy Component preventivamente y purga las colisiones jerárquicas con el nodo nativo Terminate Bodies Below.

Armas suspendidas y sockets activos (*Bug del Arma Flotante*)

- **Problema:** Al amputar la mano o el brazo de un NPC, el arma continuaba flotando en el aire siguiendo las animaciones del esqueleto original.

- **Causa raíz:** El uso de la función Hide Bone By Name escala la geometría a cero para ocultarla, pero mantiene activos y funcionales los *sockets* del esqueleto en el espacio tridimensional.
- **Solución aplicada:** Se introdujo una condición de desprendimiento: al detectarse daño crítico o mutilación en articulaciones de extremidades superiores, se ejecuta Detach From Component sobre el actor del arma y se activa Set Simulate Physics, forzando al objeto a responder a la gravedad y caer al suelo de forma independiente.

Instanciación cíclica por impactos repetidos

- **Problema:** Al recibir impactos de bala continuos sobre áreas corporales ya mutiladas, el sistema continuaba instanciando fragmentos y muñones adicionales de manera infinita.
- **Causa raíz:** La caja de colisión del canal de daño continuaba reportando impactos balísticos en los nodos articulares remanentes.
- **Solución aplicada:** Se integró un arreglo en memoria (HuesosYaDestruídos). Previo a procesar cualquier corte, una compuerta condicional valida mediante Contains si el identificador del hueso ya figura en la lista de exclusión, cancelando de inmediato cualquier cálculo redundante.

5.3. Renderizado y proyección espacial de efectos

Desbordamiento de volumen en proyección de Decals (*Decal Infinito*)

- **Problema:** Al impactar un disparo contra un muro, la mancha de sangre atravesaba la estructura geométrica y se proyectaba de forma indeseada en las paredes de habitaciones contiguas.
- **Causa raíz:** El volumen de proyección (*Bounding Box*) del componente Decal mantenía una escala longitudinal extensa por defecto a lo largo del eje local XX .
- **Solución aplicada:** Se limitó paramétricamente la dimensión del eje XX en el pin Decal Size dentro del nodo Spawn Decal at Location (fijándolo en 10 unidades frente a 50 en YY y ZZ), confinando la caja de proyección a la superficie inmediata del muro.

Conflictos de profundidad y parpadeo visual (*Z-Fighting*)

- **Problema:** Impactos reiterados en una misma coordenada espacial generaban parpadeos y manchas visualmente corruptas en modo DBuffer Translucent.
- **Causa raíz:** Coincidencia coplanar de múltiples texturas en el mismo valor de profundidad sobre el búfer gráfico.

- **Solución aplicada:** Se aplicó un factor de variación angular asistido por el nodo Random Float in Range (0.0° a 360.0°) sobre el eje de rotación (*Roll*), complementado por un micro-desplazamiento vectorial a lo largo de la normal invertida del impacto para evitar la superposición geométrica idéntica.

Contaminación de texturas y conflictos de trazado

- **Problema:** Los decals se imprimían sobre el cuerpo del jugador, armas y NPCs móviles, o se proyectaban en el aire al colisionar con cápsulas invisibles.
- **Causa raíz:** Uso del canal genérico Visibility para el trazado de decals y ausencia de filtros de recepción en las mallas de las entidades.
- **Solución aplicada:** Se desmarcó la casilla Receives Decals en los componentes de malla estática y esquelética de personajes y armas. Adicionalmente, se creó el canal dedicado Sangre_Entorno, configurado para detenerse exclusivamente ante colisiones de tipo WorldStatic e ignorar cualquier entidad física dinámica.

5.4. Concurrencia, control de flujo y sincronización en IA

Condición de carrera e inicialización nula (Accessed None en BeginPlay)

- **Problema:** El motor lanzaba errores críticos de referencia nula al iniciar la simulación, bloqueando la activación de la Inteligencia Artificial.
- **Causa raíz:** El peón físico (BP_Enemigo_Definitivo) intentaba invocar y ejecutar el árbol de comportamiento (Run Behavior Tree) desde su propio evento BeginPlay, antes de que el controlador nativo (AAIController) completara su inicialización en memoria.
- **Solución aplicada:** Se reestructuró el flujo creando la clase derivada BP_CerebroEnemigo y trasladando la llamada de inicialización del árbol de comportamiento al evento Event OnPossess, garantizando que la lógica cognitiva solo se active cuando la posesión del peón esté confirmada.

Excepciones por daño simultáneo masivo (*Overkill*)

- **Problema:** Ráfagas de disparos rápidos o impactos críticos en la cabeza provocaban congelamientos intermitentes y reportes de Accessed None durante la muerte del NPC.
- **Causa raíz:** El flujo de daño intentaba ejecutar la desconexión del controlador (UnPossess) múltiples veces dentro de un mismo fotograma de simulación.
- **Solución aplicada:** Se integró un macro de validación Is Valid antes de interactuar con el componente cerebral (Brain Component) o el controlador,

neutralizando llamadas de desconexión duplicadas y redirigiendo de inmediato los impactos posteriores directamente al cálculo físico de la malla en *Ragdoll*.

Acoplamiento rígido de armamento entre Jugador y NPC

- **Problema:** Al disparar o recargar un NPC, la animación de retroceso o recarga se ejecutaba sobre el personaje del jugador humano.
- **Causa raíz:** La clase BP_ArmaBase utilizaba el nodo estático Get Player Character (Index 0) de forma fija para invocar efectos y montajes de animación.
- **Solución aplicada:** Se desacopló la clase base de armamento eliminando nodos estáticos e integrando un parámetro de entrada universal denominado Tirador (Actor Object Reference). El sistema valida dinámicamente si el portador es un jugador o una IA antes de notificar a los respectivos grafos de animación.

Colapso de ejes en rotación procedimental de cuello (*Gimbal Lock*)

- **Problema:** Deformaciones severas en la columna y cuello del modelo 3D al intentar orientar la mirada del guardia hacia puntos de interés durante la patrulla.
- **Causa raíz:** Discrepancia matemática entre las matrices de rotación global del nodo Look At en el *AnimGraph* y los ejes locales del esqueleto (donde el hueso de la columna utilizaba el eje Y como vector ascendente nativo).
- **Solución aplicada:** Se descartó la manipulación angular directa en el *AnimGraph*. La orientación de vigilancia se transfirió al controlador de IA mediante Set Focal Point, permitiendo que el motor interpole la cápsula física completa mediante el Rotation Rate a 150°/s de forma cinemática y estable.

6. ESTADO ACTUAL Y PRÓXIMOS PASOS (ROADMAP FASE B)

El cierre del Módulo A deja operativa la infraestructura básica de percepción, toma de decisiones, locomoción y combate táctico en un entorno controlado. La Fase B abordará la transición desde comportamientos basados en puntos fijos hacia una toma de decisiones contextual dinámica en el entorno, la integración funcional de estados críticos de salud y el saneamiento de la deuda técnica sensorial.

6.1. Hitos consolidados del Módulo A

Componente	Implementación Actual	Estado
Controlador Base (C++)	Subclase de AAIController con UAIPerceptionComponent (vista, oído y daño).	Completado
Memoria de Trabajo	BB_MemoriaEnemigo desacoplado por instancia con variables de estado y vectores de última posición.	Completado
Árbol de Decisión	Jerarquía de seis ramas en BT_CerebroEnemigo con interrupciones en tiempo de ejecución (Observer Aborts).	Completado
Combate y Puntería	Rastreo asíncrono con Set Focus, control de cadencia desacoplado y balística desde el socket del cañón.	Completado
Navegación Pasiva	Patrullaje paramétrico en editor y barrido visual procedural interpolado mediante Set Focal Point.	Completado

6.2. Roadmap Fase B: Arquitectura avanzada y optimización técnica

Integración del Environment Query System (EQS) para coberturas y flanqueo

- **Generación de cuadrículas espaciales (EQS Grids):** Reemplazar las coordenadas prefijadas por generadores de puntos de prueba alrededor del NPC y del jugador.
- **Pruebas de visibilidad y distancia:** Configurar pruebas ponderadas (Trace Test hacia el jugador) para filtrar posiciones que bloqueen la línea de visión directa, identificando geometrías protectoras válidas en tiempo real.
- **Cálculo de maniobras de flanqueo:** Diseñar consultas que evalúen el ángulo respecto al vector frontal del jugador, priorizando rutas que permitan aproximaciones laterales o traseras sin exponer al NPC en campo abierto.

Implementación funcional de la rama de Agonía (Mecánica de extremidades comprometidas)

- **Activación de comportamiento por desmembramiento:** Vincular de forma efectiva la variable booleana Pierna_Destruida del Blackboard (modificada

desde el componente BPC_Salud tras registrar amputaciones inferiores) con la rama de máxima prioridad del árbol.

- **Integración en *AnimGraph* y locomoción reducida:** Suspender el ciclo estándar de caminado y trote; forzar la transición hacia animaciones de arrastre o incapacidad física mediante montajes específicos en el grafo de animación.
- **Restricción de capacidades ofensivas:** Suprimir tareas de asalto o cobertura activa mientras el NPC esté en estado de agonía, limitando sus acciones a defensa desesperada en el suelo con armas cortas o neutralización definitiva.

Filtrado sensorial contextual y optimización de percepción auditiva

- **Inhibición de estímulos auditivos en combate activo:** Corregir la saturación de procesamiento sensorial implementando un filtro de estado en C++. Si el NPC ya cuenta con confirmación visual del objetivo (`JugadorVisto = True`) o se encuentra en estado de alerta máxima, el sistema desestimaré o reducirá la prioridad de las alertas acústicas circundantes, evitando oscilaciones erráticas de atención o interrupciones en las secuencias de disparo.
- **Persistencia y decaimiento del estado de alerta:** Ajustar los temporizadores de pérdida de estímulo para que la transición de combate a búsqueda no limpie abruptamente el foco táctico del peón.

Integración de la interfaz de equipos (*Generic Team Agent Interface*)

- **Clasificación nativa de bandos:** Implementar la interfaz nativa `IGenericTeamAgentInterface` en la clase del controlador C++ para gestionar identificadores de equipo (*Team IDs*).
- **Sustitución de filtros por etiquetas:** Reemplazar las comprobaciones manuales por `ActorHasTag` por la matriz nativa de actitudes del motor (`ETeamAttitude::Hostile`, `Friendly`, `Neutral`), optimizando el filtrado de percepción directamente en el pipeline del motor sensorial.

Sincronización cinemática de la mirada (*Aim Offsets* y espacio de componente)

- **Corrección definitiva del seguimiento de cabeza:** Reintegrar el seguimiento visual dinámico hacia puntos de interés resolviendo el fallo de orientación articular previa mediante un espacio de interpolación de puntería (*Aim Offset*).
- **Aislamiento articular:** Restringir las deformaciones exclusivamente a las articulaciones `neck_01` y `head`, garantizando que operen en espacio de malla local para eliminar distorsiones en la columna dorsal y mantener la cápsula física correctamente orientada.